UNIVERSIDADE POSITIVO
CAMPUS PRAÇA OSÓRIO
CURSO ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: ARQUITETURA MVVM, REATIVIDADE COM FLOW E PADRÃO REPOSITORY

Curitiba
2026

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: ARQUITETURA MVVM, REATIVIDADE COM FLOW E PADRÃO REPOSITORY

Relatório técnico apresentado à disciplina de Desenvolvimento de Aplicativos Móveis do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Positivo, como requisito parcial à avaliação da atividade prática.

Prof. André Jeller

Curitiba
2026

RESUMO

O presente relatório técnico documenta a evolução arquitetural final do aplicativo móvel **PlayDex**, desenvolvida para a transição do modelo do Trabalho 3 para uma arquitetura robusta baseada no padrão **MVVM (Model-View-ViewModel)** com **Repository**, **UiState** e fluxos de dados assíncronos e reativos (**Kotlin Flow** / **StateFlow**). A reestruturação contemplou a eliminação de chamadas diretas ao banco de dados Room nas views Composable, o isolamento da camada de dados por meio de Repositories dedicados (`ItemFavoritoRepository` e `ItemPesquisaRepository`), o encapsulamento de estado imutável nas telas de Favoritos e Pesquisa, e a implementação do padrão *Factory* para injeção de dependências nos ViewModels. Com essas alterações, o sistema alcançou desacoplamento completo entre a interface gráfica e o banco SQLite local, garantindo reatividade automática perante alterações no banco de dados e manutenção da integridade do estado da interface.

Palavras-chave: MVVM; StateFlow; Kotlin Flow; Repository Pattern; Jetpack Compose; Room Database; Android.

SUMÁRIO

1. COMO ESTAVA O PROJETO NO TRABALHO 3 E O QUE MUDOU NA REORGANIZAÇÃO..... 14
2. ORGANIZAÇÃO EM PACOTES (ESTRUTURA DE PASTAS)..... 16
2.1 Detalhamento dos Pacotes..... 16
3. FINALIDADE DO REPOSITORY E O QUE ELE RESOLVEU NA ARQUITETURA 18
3.1 O que o Repository resolveu..... 18
3.2 Comparação do Código (Antes vs. Depois)..... 18
3.2.1 Antes (Acesso direto ao DAO no ViewModel)..... 18
3.2.2 Depois (Com Repository Pattern)..... 19
4. COMPOSIÇÃO E JUSTIFICATIVA DOS UISTATES..... 20
4.1 FavoritosUIState..... 20
4.2 PesquisaUiState..... 20
4.3 Padrão de Encapsulamento com StateFlow (Backing Property)..... 21
5. PROBLEMAS ENCONTRADOS E SOLUÇÕES ADOTADAS..... 22
5.1 Conflito com suspend em Funções com Retorno Flow no DAO..... 22
5.2 Instanciação de ViewModel com Parâmetros no Construtor..... 22
5.3 Incompatibilidade de Dependências Gradle (Erro de SDK e AGP)..... 22
5.4 Ambiguidade no Import da Extensão items do Compose..... 23
6. DIFICULDADES INDIVIDUAIS E EM GRUPO E RESOLUÇÕES ADOTADAS.... 24
6.1 Compreensão e Desacoplamento de Responsabilidades..... 24
6.2 Criação e Organização de Arquivos Específicos do MVVM..... 24
7. DEMONSTRAÇÃO DO APLICATIVO EM FUNCIONAMENTO..... 26
7.1 Demonstração Audiovisual..... 26
8. CONSIDERAÇÕES FINAIS..... 27

1. COMO ESTAVA O PROJETO NO TRABALHO 3 E O QUE MUDOU NA REORGANIZAÇÃO

No **Trabalho 3**, embora o banco de dados relacional SQLite/Room e os ViewModels (estendendo **`AndroidViewModel`**) já estivessem integrados à aplicação, existiam limitações arquiteturais importantes:

1. **Chamadas diretas ao DAO pelas ViewModels e Funções Utilitárias:** A lógica de acesso ao banco ficava exposta no ViewModel ou em funções soltas (**`OperacoesBanco.kt`**), misturando responsabilidades de busca, tratamento de exceção e chamadas diretas ao Room.
2. **Consultas Estáticas (Sem Reatividade Nativa):** As consultas de listagem (**`buscarFavoritos()`** e **`buscarItens()`**) retornavam listas estáticas simples (**`List<T>`**). Isso exigia que cada inserção ou deleção chamasse manualmente uma função de recarregamento para atualizar a interface.
3. **Gerenciamento de Estado Fragmentado:** O estado da interface nas telas era gerenciado por múltiplas variáveis soltas (**`mutableStateOf`** e **`mutableStateListOf`**), dificultando a previsibilidade da UI.
4. **Dependência Direta do Contexto da Aplicação:** Os ViewModels herdavam de **`AndroidViewModel`** para instanciar o **`AppDatabase`** internamente.

Na **Reorganização Atual (Trabalho 4)**, o projeto passou pelas seguintes transformações estruturais:

- **Adoção do Padrão Repository:** Criaram-se as classes **`ItemFavoritoRepository`** e **`ItemPesquisaRepository`**, tornando-as a **única fonte de verdade e única porta de entrada** para a camada de dados.
- **Reatividade com Flow:** Os métodos de consulta nos DAOs passaram a retornar **`Flow<List<T>>`**. O Room agora notifica e emite uma nova lista automaticamente sempre que a tabela sofre alterações (CRUD).
- **Encapsulamento do Estado com UiState e StateFlow:** O estado de cada tela reativa foi agrupado em *data classes* imutáveis (**`FavoritosUIState`** e **`PesquisaUiState`**), expostos via **`StateFlow`** e manipulados estritamente através

da *backing property* (**`_uiState`** privado e mutável, **`uiState`** público e somente leitura).

- **Desacoplamento e Injeção via Factory:** Os ViewModels passaram a herdar de **`ViewModel`** puro e a receber seus respectivos Repositories no construtor por meio de **`ViewModelProvider.Factory`**.
- **Views Puramente Passivas ("Burras"):** As telas **`@Composable`** não possuem referências diretas ao **`DAO`**, **`CoroutineScope`** ou **`Dispatchers.IO`**. Elas apenas observam o **`uiState`** via **`collectAsStateWithLifecycle()`** e delegam eventos ao ViewModel.

2. ORGANIZAÇÃO EM PACOTES (ESTRUTURA DE PASTAS)

Para atender ao princípio de separação de responsabilidades (SoC) e manter a arquitetura limpa e escalável, o projeto foi reorganizado nos pacotes principais abaixo:

```
├── MainActivity.kt
├── data/
│   ├── local/
│   │   ├── dao/
│   │   │   ├── FavoritoDao.kt
│   │   │   └── PesquisaDAO.kt
│   │   ├── entity/
│   │   │   ├── ItemFavorito.kt
│   │   │   └── ItemPesquisa.kt
│   │   └── AppDatabase.kt
│   └── repository/
│       ├── ItemFavoritoRepository.kt
│       ├── ItemPesquisaRepository.kt
│       └── OperacoesBanco.kt
├── navigation/
│   └── Rotas.kt
├── ui/
│   ├── state/
│   │   ├── FavoritosUIState.kt
│   │   └── PesquisaUiState.kt
│   ├── telas/
│   │   ├── TelaAdicionarFavorito.kt
│   │   ├── TelaConfiguracoes.kt
│   │   ├── TelaDadosJogo.kt
│   │   ├── TelaDetalhesFavorito.kt
│   │   ├── TelaFavoritos.kt
│   │   ├── TelaPerfilUsuario.kt
│   │   ├── TelaPesquisa.kt
│   │   ├── TelaPrincipal.kt
│   │   └── TelaReview.kt
│   └── theme/
│       ├── Color.kt
│       ├── Theme.kt
│       └── Type.kt
└── viewModel/
    ├── FavoritosViewModel.kt
    └── PesquisaViewModel.kt
```

2.1 Detalhamento dos Pacotes

- **`data.local` (`dao`, `entity` e `AppDatabase`):** Armazena a estrutura do banco SQLite/Room, contendo as entidades mapeadas (`@Entity`), as interfaces DAO (`@Dao`) e a classe singleton do banco (`@Database`).

- **`data.repository`:** Contém as classes que abstraem a origem dos dados e servem de ponte exclusiva para o ViewModel.
- **`ui (state, telas, theme)`:** Reúne a camada visual. O subpacote `state` guarda as estruturas de dados do `UiState`, `telas` contém as funções `@Composable` da interface e `theme` gerencia o estilo visual do aplicativo.
- **`viewModel`:** Contém as classes que gerenciam a lógica de negócios, controlam o `StateFlow` e contêm as `ViewModelFactory` para instanciação dos ViewModels.
- **`navigation`:** Centraliza a definição das rotas de navegação do aplicativo.

3. FINALIDADE DO REPOSITORY E O QUE ELE RESOLVEU NA ARQUITETURA

O **Repository** foi introduzido para atuar como uma **camada de abstração intermediária entre a camada de dados (Room/DAO) e a camada de apresentação (ViewModel)**.

3.1 O que o Repository resolveu

1. **lsolamento da Origem dos Dados:** O ViewModel não precisa saber se os dados vêm de um banco de dados relacional local (Room), de uma API REST ou de um cache em memória. Caso a fonte de dados mude no futuro, apenas a classe Repository precisa ser alterada, mantendo ViewModels e Views intactos.
2. **Tratamento de Exceções e Segurança:** Centraliza os blocos `try-catch` e operações de log nas mutações de banco de dados, evitando que falhas de E/S quebrem a execução do aplicativo.
3. **Eliminação de Dependências do Room na UI:** Garante que o ViewModel dependa de uma abstração de dados em vez de gerenciar diretamente instâncias do `AppDatabase` ou da interface `DAO`.

3.2 Comparação do Código (Antes vs. Depois)

3.2.1 Antes (Acesso direto ao DAO no ViewModel)

```
// FavoritosViewModel.kt (Trabalho 3) - Sem Repository
class FavoritosViewModel(application: Application) :
AndroidViewModel(application) {
    private val db = AppDatabase.getDatabase(application)
    private val favoritoDao = db.favoritoDao()

    fun carregarFavoritos() {
        viewModelScope.launch(Dispatchers.IO) {
            val lista = buscarFavoritos(favoritoDao)
            withContext(Dispatchers.Main) { favoritos = lista }
        }
    }
}
```

3.2.2 Depois (Com Repository Pattern)

```
// ItemFavoritoRepository.kt (Trabalho 4)
class ItemFavoritoRepository(private val favoritoDao: FavoritoDao) {

    fun buscarTodosFavoritos(): Flow<List<ItemFavorito>> =
favoritoDao.buscarFavoritos()

    suspend fun inserirFavorito(item: ItemFavorito) {
        try {
            favoritoDao.inserir(item)
        } catch (e: Exception) {
            Log.e("Repository", "Erro ao adicionar favorito: ${e.message}")
        }
    }
    // ...
}

// FavoritosViewModel.kt (Trabalho 4) - ViewModel limpo recebendo o Repository
class FavoritosViewModel(private val repository: ItemFavoritoRepository) :
ViewModel() {
    // ViewModel interage apenas com o repository
}
```

4. COMPOSIÇÃO E JUSTIFICATIVA DOS UISTATES

Para garantir o **Fluxo de Dados Unidirecional (UDF)**, centralizou-se o estado das telas reativas em *data classes* imutáveis.

4.1 `FavoritosUIState`

```
data class FavoritosUIState(
    val listaFavoritos: List<ItemFavorito> = emptyList(),
    val filtroSelecionado: String = "Todos",
    val adicionandoFavorito: Boolean = false,
    val favoritoSelecionado: ItemFavorito? = null,
    val resultadoBuscaId: ItemFavorito? = null
)
```

Justificativa dos campos:

- **`listaFavoritos`**: Guarda a lista de jogos favoritados emitida em tempo real pelo `Flow` do Room.
- **`filtroSelecionado`**: Controla qual filtro de exibição ("Todos", "Joguei", "Quero jogar", "Gostei") está ativo.
- **`adicionandoFavorito`**: Flag booleana que controla a visibilidade da tela/modal de cadastro.
- **`favoritoSelecionado`**: Armazena a referência do item clicado para exibição/edição na tela de detalhes.
- **`resultadoBuscaId`**: Mantém o resultado retornado ao realizar uma consulta de favorito por ID.

4.2 `PesquisaUiState`

```
data class PesquisaUiState(
    val textoPesquisa: String = "",
    val historico: List<ItemPesquisa> = emptyList()
)
```

Justificativa dos campos:

- **`textoPesquisa`**: Mantém o texto digitado na barra de pesquisa sem precisar de variáveis mutáveis soltas.

- **`historico`**: Representa a lista de termos buscados recuperados em tempo real do banco de dados.

4.3 Padrão de Encapsulamento com `StateFlow` (Backing Property)

Ambos os ViewModels adotam o padrão rigoroso de proteção de estado:

```
// Estado privado e mutável: apenas o ViewModel pode alterar internamente
private val _uiState = MutableStateFlow(FavoritosUIState())

// Estado público e imutável: a View apenas observa (somente leitura)
val uiState: StateFlow<FavoritosUIState> = _uiState.asStateFlow()
```

5. PROBLEMAS ENCONTRADOS E SOLUÇÕES ADOTADAS

Durante o processo de refatoração para a arquitetura MVVM e reatividade com Flow, a equipe enfrentou e resolveu quatro problemas técnicos principais:

5.1 Conflito com `suspend` em Funções com Retorno `Flow` no DAO

- **Problema:** Inicialmente, a função **`buscarFavoritos()`** na DAO foi declarada como **`suspend fun buscarFavoritos(): Flow<List<ItemFavorito>>`**. O Room acusou erro de compilação pois fluxos contínuos (**`Flow`**) já são assíncronos por natureza e não devem utilizar a palavra-chave **`suspend`**.
- **Solução:** Removeu-se o modificador **`suspend`** da assinatura dos métodos que retornam **`Flow`** nas interfaces **`FavoritoDao`** e **`PesquisaDAO`**.

```
// Correção na FavoritoDao.kt
@Query("SELECT * FROM tabela_favoritos ORDER BY id DESC")
fun buscarFavoritos(): Flow<List<ItemFavorito>> // Sem 'suspend'
```

5.2 Instanciação de ViewModel com Parâmetros no Construtor

- **Problema:** Ao passar o **ItemFavoritoRepository** no construtor do **FavoritosViewModel**, o framework do Android deixou de conseguir instanciar o ViewModel diretamente via **viewModel()** nas telas Compose, gerando a exceção **IllegalArgumentException: Unknown ViewModel class**.
- **Solução:** Implementou-se a interface **ViewModelProvider.Factory** para ensinar o Android como criar o ViewModel injetando o Repository:

```
class FavoritosViewModelFactory(
    private val repository: ItemFavoritoRepository
) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(FavoritosViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return FavoritosViewModel(repository) as T
        }
        throw IllegalArgumentException("Classe ViewModel desconhecida")
    }
}
```

5.3 Incompatibilidade de Dependências Gradle (Erro de SDK e AGP)

- **Problema:** Ao adicionar as bibliotecas **`lifecycle-viewmodel-compose`** e **`lifecycle-runtime-compose`**, a resolução automática baixou a versão **`2.11.0`**, que exigia o Android SDK 37 e o Android Gradle Plugin 9.1.0, gerando falhas no build do projeto compilado no SDK 36.
- **Solução:** Fixou-se a versão das bibliotecas de ciclo de vida do Compose em **`2.8.7`** no arquivo **`build.gradle.kts`**, garantindo total compatibilidade com o ambiente configurado.

5.4 Ambiguidade no Import da Extensão `items` do Compose

- **Problema:** Ao renderizar a **`LazyColumn`** na **`TelaFavoritos.kt`**, o compilador apresentou sublinhado vermelho na chave **`key = { item -> item.id }`** devido à falta do import correto da extensão de listas do Compose.
- **Solução:** Adicionou-se explicitamente o import **`import androidx.compose.foundation.lazy.items`** e especificou-se o parâmetro nomeado **`items = listaFiltrada`** na chamada da **`LazyColumn`**.

6. DIFICULDADES INDIVIDUAIS E EM GRUPO E RESOLUÇÕES ADOTADAS

Durante o processo de reestruturação do projeto para a arquitetura exigida, a principal dificuldade enfrentada pela equipe residiu na **desconstrução do modelo anterior para a separação rigorosa das responsabilidades dos ViewModels em arquivos específicos**.

6.1 Compreensão e Desacoplamento de Responsabilidades

No estágio do Trabalho 3, a equipe já utilizava ViewModels; contudo, esses arquivos concentravam tanto o estado da tela quanto chamadas diretas ao banco de dados e instâncias do Room. Individualmente, o maior desafio foi entender conceitualmente onde terminava a responsabilidade de cada componente:

- **O problema:** O grupo sentia dificuldade em identificar como retirar as variáveis mutáveis de estado (`mutableStateOf`) de dentro das ViewModels e repassá-las para arquivos dedicados de `UiState` sem quebrar o fluxo de renderização das telas Jetpack Compose.
- **A resolução:** Compreendeu-se que o estado visual deve ser tratado como um objeto imutável único (`data class`), e a ViewModel passa a ser apenas a manipuladora desse objeto via `MutableStateFlow`.

6.2 Criação e Organização de Arquivos Específicos do MVVM

Em grupo, a transição exigiu a criação de múltiplos arquivos para fluxos que antes ficavam concentrados em poucas classes. A criação de arquivos dedicados para os Repositories (`ItemFavoritoRepository`, `ItemPesquisaRepository`), para os estados (`FavoritosUIState`, `PesquisaUiState`) e para as fábricas (`FavoritosViewModelFactory`, `PesquisaViewModelFactory`) gerou dúvidas iniciais sobre como conectar todas as pontas:

- **O problema:** A interligação entre a View (que precisava da Factory), a Factory (que precisava do Repository) e o Repository (que precisava do DAO) parecia burocrática e causou erros de instanciação durante os primeiros testes.
- **A resolução:** A equipe mapeou visualmente o fluxo de injeção de dependências: a Tela Compose obtém o contexto local, instancia o DAO via `AppDatabase`, entrega ao Repository, que por sua vez é passado à Factory para que o Android forneça o ViewModel configurado com segurança.

Essa etapa de padronização, embora desafiadora, permitiu ao grupo compreender na prática os benefícios do desacoplamento, resultando em um código modular, mais fácil de manter e testar.

7. DEMONSTRAÇÃO DO APLICATIVO EM FUNCIONAMENTO

As funcionalidades do aplicativo **PlayDex** foram testadas e validadas quanto à persistência local e reatividade da interface.

7.1 Demonstração Audiovisual

O vídeo demonstrativo completo do aplicativo em execução (apresentando as rotas, inclusão de favoritos, pesquisas, filtros dinâmicos e persistência do estado ao reiniciar o aplicativo) pode ser acessado no link a seguir:

- **Vídeo do Funcionamento:** Vídeo de demonstração.mp4

8. CONSIDERAÇÕES FINAIS

A realização deste trabalho permitiu consolidar a transição do aplicativo **PlayDex** para uma aplicação móvel com arquitetura profissional de alta qualidade. A substituição do modelo do Trabalho 3 por uma estrutura embasada no padrão **MVVM**, respaldada por **Repositories**, **UiStates imutáveis** e **Kotlin Flow**, eliminou o acoplamento entre as telas Compose e o banco de dados Room. A aplicação agora oferece reatividade automática em tempo real, mantendo a estabilidade e a retenção de dados através das mudanças de ciclo de vida do Android com total previsibilidade.